

Article

Comprehensive Examination of Unrolled Networks for Solving Linear Inverse Problems [†]

Yuxi Chen ¹, Xi Chen ², Arian Maleki ³ and Shirin Jalali ^{2,*}

¹ Department of Statistics and Data Science, Carnegie Mellon University, Pittsburgh, PA 15213, USA; ericc3@andrew.cmu.edu

² Electrical and Computer Engineering Department, Rutgers University, New Brunswick, NJ 08854, USA; xi.chen15@rutgers.edu

³ Department of Statistics, Columbia University, New York, NY 10027, USA; arian@stat.columbia.edu

* Correspondence: shirin.jalali@rutgers.edu

[†] This article is a revised and expanded version of a paper entitled “Deep Memory Unrolled Networks for Solving Imaging Linear Inverse Problems”, which will be presented at the 15th International Conference on Sampling Theory and Applications in Vienna, Austria in 28 July 2025.

Abstract

Unrolled networks have become prevalent in various computer vision and imaging tasks. Although they have demonstrated remarkable efficacy in solving specific computer vision and computational imaging tasks, their adaptation to other applications presents considerable challenges. This is primarily due to the multitude of design decisions that practitioners working on new applications must navigate, each potentially affecting the network’s overall performance. These decisions include selecting the optimization algorithm, defining the loss function, and determining the deep architecture, among others. Compounding the issue, evaluating each design choice requires time-consuming simulations to train, fine-tune the neural network, and optimize its performance. As a result, the process of exploring multiple options and identifying the optimal configuration becomes time-consuming and computationally demanding. The main objectives of this paper are (1) to unify some ideas and methodologies used in unrolled networks to reduce the number of design choices a user has to make, and (2) to report a comprehensive ablation study to discuss the impact of each of the choices involved in designing unrolled networks and present practical recommendations based on our findings. We anticipate that this study will help scientists and engineers to design unrolled networks for their applications and diagnose problems within their networks efficiently.

Keywords: compressed sensing; deep unrolled networks; computational imaging



Academic Editor: Ali Mohammad-Djafari

Received: 2 June 2025

Revised: 30 August 2025

Accepted: 1 September 2025

Published: 3 September 2025

Citation: Chen, Y.; Chen, X.; Maleki, A.; Jalali, S. Comprehensive Examination of Unrolled Networks for Linear Inverse Problems. *Entropy* **2025**, *27*, 929. <https://doi.org/10.3390/e27090929>

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the Creative Commons

Attribution (CC BY) license

(<https://creativecommons.org/licenses/by/4.0/>).

1. Unrolled Networks for Linear Inverse Problems

In many imaging applications, ranging from magnetic resonance imaging (MRI) and computational tomography (CT scan) to seismic imaging and nuclear magnetic resonance (NMR), the measurement process can be modeled in the following way:

$$y = Ax^* + w.$$

In the above equation, $y \in \mathbb{R}^m$ represents the collected measurements and $x^* \in \mathbb{R}^n$ denotes the vectorized image that we aim to capture. The matrix $A \in \mathbb{R}^{m \times n}$ represents the forward operator or measurement matrix of the imaging system, which is typically known exactly or with some small error. Finally, w represents the measurement noise, which is not known,

but some information about its statistical properties (such as the approximate shape of the distribution) may be available.

Recovering x^* from the measurement y has been extensively studied, especially in the last 15 years after the emergence of the field of compressed sensing [1–4]. In particular, between 2005 and 2015, many successful algorithms were proposed to solve this problem, including Denoising-based AMP [5,6], Plug-and-Play Priors [7], compression-based recovery [8,9], and Regularization by Denoising [10].

Inspired by the successful application of neural networks, many researchers have started exploring the application of neural networks to solve linear inverse problems [11–38]. The original networks proposed for this goal were deep networks that combined convolutional and fully connected layers [12,39]. The idea was that we feed y or $A^T y$ to a network and then expect the network to eventually return x^* .

While these methods performed reasonably well in some applications, in many cases, they underperformed the more classical non-neural network-based algorithms and simultaneously required computationally demanding training. Some of the challenges that these networks face are as follows:

- **Size of the measurement matrix:** The forward model under matrix A has $m \times n$ elements. Even if the image is small, we may have $n = 256 \times 256$ and $m = 128 \times 128$. This means that the measurement matrix may have more than 1 billion elements. Consequently, an effective deep learning-based recovery algorithm may need to memorize the elements or learn the structural properties of A to be able to reconstruct x^* from y . This means that the neural network itself should ideally have many more parameters. Not only is the training of such networks computationally very demanding but also, within the computational limits of the work that has been conducted so far, end-to-end networks have not been very successful.
- **Changes in the measurement matrix:** Another issue that is faced by such large networks is that usually one needs to redesign a network and train a model specific to each measurement matrix. Each network often suffers from poor generalizability to even small changes in the matrix entries.
- **Forward model inconsistency:** It should also be noted that these end-to-end neural networks do not solve the inverse problem in the mathematical sense—they learn approximate mappings without guaranteeing consistency with the forward model $y = Ax$, as demonstrated in CT imaging applications [40].

To address the issues faced by such deep and complex networks in solving inverse problems, and inspired by iterative algorithms for solving convex and non-convex problems, a category of networks known as unrolled networks has emerged [41,42]. To understand the motivation behind these unrolled networks, we consider the hypothetical situation in which all images of interest belong to a set $\mathcal{C} \subset \mathbb{R}^n$. Under this assumption, one way to recover the image x^* from the measurements y is to find

$$\arg \min_{x \in \mathcal{C}} \|y - Ax\|_2^2. \quad (1)$$

One method to solve this optimization problem is via projected gradient descent (PGD), which uses the following iterative steps:

$$\begin{aligned} \tilde{x}^i &= x^i + \mu A^T (y - Ax^i), \\ x^{i+1} &= P_{\mathcal{C}}(\tilde{x}^i). \end{aligned} \quad (2)$$

where x^i is the estimate of x^* in iteration i , μ is the step size (learning rate), and $P_{\mathcal{C}}$ denotes the projection onto the set $\mathcal{C}$. Figure 1 shows a diagram of the projected gradient descent algorithm.

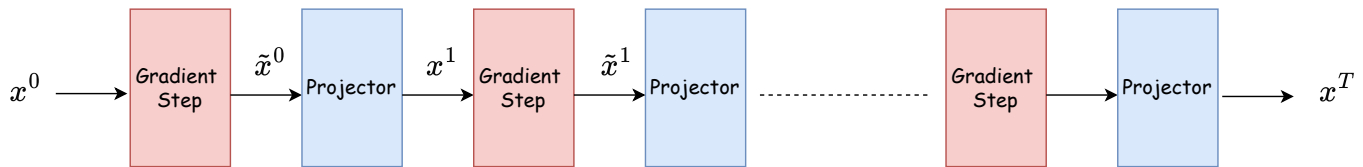


Figure 1. Diagram of projected gradient descent. Starting with $x^0 = 0$, the i^{th} gradient step performs the operation $\tilde{x}^i = x^i + \mu A^T(y - Ax^i)$, and the i^{th} projector unit performs $x^{i+1} = P_{\mathcal{C}}(\tilde{x}^i)$.

One of the challenges in using PGD for linear inverse problems is that the set $\mathcal{C}$ is unknown, and hence $P_{\mathcal{C}}$ is also not known. For example, $\mathcal{C}$ can represent all natural images of a certain size and $P_{\mathcal{C}}(\cdot)$ a projection onto that space. Researchers have considered ideas such as using state-of-the-art image compression algorithms and image denoising algorithms for the projection step [6–10,43], and have more recently adopted neural networks as a promising approach [11,18,44]. In these formulations, all the projectors in Figure 1 are replaced with neural networks (usually, the same neural network architecture can be used at different steps). There are several analytical and computational benefits to such an approach:

- We do not require a heuristically pre-designed denoiser or compression algorithm to act as the projector $P_{\mathcal{C}}(\cdot)$. Instead, we can deploy a training set to train the networks and optimize their performance. This enables the algorithm to potentially achieve better performance.
- Although projected gradient descent analytically employs the same projection operator at every iteration, once we replace them with neural networks, we do not need to impose the constraint of all the neural networks' learned parameters being the same. In fact, giving more freedom can enable us to train the networks more efficiently and, at the same time, improve performance.
- Using neural networks enables greater flexibility and can integrate with a wide range of iterative optimization algorithms. For instance, although the above formulation of the unrolled network follows from projected gradient descent, one can also design unrolled networks using a wide range of options, including heavy-ball methods and message passing algorithms.

The above formulation of combining projected gradient descent with neural networks belongs to a family called *deep unrolled networks* or *deep unfolded networks*, which is a class of neural network architectures that integrate iterative model-based optimization algorithms with data-driven deep learning approaches [29,42,45]. The central idea is to “unroll” an iterative optimization algorithm a given number of times, where each iteration replaces the traditional mapping or projection operator with a neural network. The parameters of this network are typically learned end to end. By enabling parameters to be learnable within this framework, unrolled networks combine the interpretability and convergence properties of traditional algorithms with the adaptivity and performance of deep learning models. Due to their efficacy, these networks have been widely used in solving linear inverse problems.

The unrolled network can also be constructed by replacing the projected gradient descent algorithm with various other iterative optimization algorithms. Researchers have explored incorporating deep learning-based projectors with a range of iterative methods, including the Alternating Direction Method of Multipliers (ADMM-Net) [11], Iterative Soft-Thresholding Algorithm (ISTA-Net) [17], Nesterov’s Accelerated First Order Method [46,47],

and approximate message passing (AMP-Net) [48]. Many of these alternatives offer faster convergence for convex optimization problems, raising the prospect of reducing the number of neural network projectors required, thereby lowering the computational complexity of both training and deploying these networks.

The remainder of this paper is organized as follows. Section 2 discusses the challenges in designing unrolled networks. Section 3 introduces our Deep Memory Unrolled Network (DeMUN), which generalizes existing unrolled algorithms. Section 4 presents our four main hypotheses with supporting experiments on loss functions, residual connections, and network complexity. Section 5 demonstrates robustness across different measurement matrices, noise levels, and image resolutions. Section 6 concludes with practical guidelines for designing effective unrolled networks.

2. Challenges in Using Unrolled Networks

As discussed above, the flexibility of unrolled networks has established them as a powerful tool for solving imaging inverse problems. However, applying these architectures to address specific inverse problems presents significant challenges to users. These difficulties stem primarily from two factors: (i) a multitude of design choices and (ii) robustness to noise, measurement matrix, and image resolution. We clarify these issues and present our approach to addressing them below.

2.1. Design Choices

The first challenge lies in the numerous design decisions that users must make when employing unrolled networks. We list some main choices below:

- **Optimization Algorithm.** In training any unrolled network, the user must decide on which iterative optimization algorithm to unroll. The choices include projected gradient descent, heavy-ball methods (such as Nesterov's accelerated first-order method), approximate message passing (AMP), the Alternating Direction Method of Multipliers (ADMM), among others. Unrolled networks trained on different optimization algorithms may lead to drastically varying performances for the task at hand.
- **Loss Function.** For any unrolled optimization algorithm, given any observation y , one produces a sequence of T projections $\{x^i\}_{i=1}^T$ (see Figure 1 for an illustration). To train the model, the convention is to define the loss function with respect to the final projection x^T using the ℓ^2 loss $\|x^T - x^*\|_2^2$ since this is usually the quantity returned by the network. However, given the non-convexity of the cost function that is used during training, there is no guarantee that the above loss function is optimal for the generalization error. For example, one could use a loss function that incorporates one or more estimates from intermediate stages, such as $\|x^T - x^*\|_2^2 + \|x^{T/2} - x^*\|_2^2$, to potentially achieve better training that provides an improved estimate of x^* . As will be shown in our simulations, the choice of the loss function has a major impact on the performance of the learned networks. Various papers have considered a wide range of loss functions for training different networks. We categorize them broadly below.
 - **Last-Layer Loss.** Consider the notations used in Figure 1. The last-layer loss evaluates the performance of the network using the following loss function:

$$\ell_{ll}(x^T, x^*) = \|x^T - x^*\|_2^2. \quad (3)$$

The last-layer loss is the most popular loss function that has been used in applications. The main argument for using this loss is that, since we only care about the final estimate and that is used as our final reconstruction, we should consider the error of the last estimate.

- **Weighted Intermediate Loss.** While the loss function above seems reasonable, some works in related fields have proposed using an intermediate loss function instead [49,50]. We define the general version of the intermediate loss function as follows:

$$\ell_{i,\omega}(x^1, x^2, \dots, x^T, x^*) = \sum_{i=1}^T \omega^{T-i} \|x^i - x^*\|_2^2, \quad (4)$$

where $\omega \in (0, 1]$. One argument that motivates the use of such loss function is that, if the predicted image after each projection is closer to the ground truth x^* , then it will help the subsequent steps to reach better solutions. The weighted intermediate loss tries to find the right balance among the accuracy of the estimates at different iterations [49]. In addition, we make the following observations:

- * When $\omega = 1$, the losses from different layers of the unrolled network are weighted equally. This means that our emphasis on the performance of the last layer is "weakened." However, this is not necessarily undesirable. As we will show in our simulations, improving the estimates of the intermediate steps will also help to improve the recovery quality of x^T .
- * As we decrease the value of $\omega \rightarrow 0$, we see that the loss function $\ell_{i,\omega}$ approaches the last-layer loss $\ell_{||}$. The choice of the ω therefore enables us to interpolate between the two cases.
- **Skip L -Layer Intermediate Loss.** Another loss function that we investigate is what we call the skip L -layer intermediate loss. This loss is similar to the loss used in Inception networks for image classification [51]. Let L be a factor of T . Then, the skip L -layer loss is given by

$$\ell_{s,L}(x^1, x^2, \dots, x^T, x^*) = \sum_{i=0}^{T/L-1} \|x^{T-iL} - x^*\|_2^2. \quad (5)$$

For instance, if $T = 15$ and $L = 3$, the skip 3-layer intermediate loss will evaluate the sum of the mean-squared errors between x^* and projections x^3, x^6, x^9, x^{12} , and x^{15} . By ranging L from 1 to T , one can again interpolate between the two loss functions $\ell_{i,1}$ and $\ell_{||}$.

- **Number of Unrolled Steps.** Practitioners also have to decide on the number of steps T to unroll for any optimization algorithm. Increasing T often comes with additional computational burdens and may also lead to overfitting. A proper choice of T can ensure that network training is not prohibitively expensive and ensure desirable levels of performance.
- **Complexity of the Neural Network.** Similar to the above, the choice of P_C also has a significant impact on the performance of the network. The options entail the number of layers or depth of the network, the activation function to use, whether or not to include residual connections, etc. If the projector is designed to have only little capacity, the unrolled network may have poor recovery. However, if the projector has excessive capacity, the network may become computationally expensive to train and prone to overfitting.

It is important to note that, after making all the design choices, users are required to conduct time-consuming, computationally demanding, and costly simulations to train the network. Consequently, users may only have the opportunity to explore a limited number of options before settling on their preferred architecture.

2.2. Robustness and Scaling

When designing unrolled networks for inverse problems, it is common to aim for robustness across a range of settings beyond the specific conditions for which the algorithm was originally designed. While an algorithm may be tailored for a particular signal type, image resolution, number of observations, or noise level, it is desirable for the network to maintain effectiveness across different settings as well.

As a simple motivating example, consider when a new imaging device has been acquired that operates using a different observation matrix. If the unrolled network previously designed has bad adaptivity and performance with respect to the current matrix, one would be required to review the entire process to determine a new batch of choices for the current setting. Therefore, ideally, one would like to have a single network structure that works well across a wide range of applications.

2.3. Our Approach for Designing Unrolled Networks

As discussed above, one faces an abundance of design choices before training and deploying an unrolled network. However, testing the performance of all the possible enumerations of these choices across a wide range of applications and datasets is computationally demanding and combinatorially prohibitive. This hinders practitioners from applying the optimal unrolled network in their problem-specific applications. To offer a more systematic way for designing such networks, we adopt the following high-level approach:

- We present the **Deep Memory Unrolled Network (DeMUN)**, where each step of the network leverages the gradient from all previous iterations. These networks encompass various existing models as special cases. The DeMUN lets the data decide on the optimal choice of algorithm to be unrolled and improves recovery performance.
- We present several hypotheses regarding important design choices that underlie the design of unrolled networks, and we test them using extensive simulations. These hypotheses allow users to avoid exploring the multitude of design choices that they have to face in practice.

These two steps allow users to bypass many design choices, such as selecting an optimization algorithm or loss function, thus simplifying the process of designing unrolled networks. We test the robustness of our hypotheses with respect to the changes in the measurement matrices and noise in the system. These robustness results suggest that the simplified design approach presented in this paper can be applied to a much wider range of systems than those specifically studied here.

3. Deep Memory Unrolled Network (DeMUN)

As discussed previously, one of the initial decisions users face when designing an unrolled network is selecting the optimization algorithm to unroll. Various optimization algorithms, including gradient descent, heavy-ball methods, and approximate message passing, have been incorporated into unrolled networks. We introduce the Deep Memory Unrolled Network (DeMUN), which encompasses many of these algorithms as special cases. At the i -th iteration in the DeMUN, the update of $\tilde{x}^i$ is given by

$$\tilde{x}^i = \alpha^i x^i + \sum_{j=0}^i \beta_j^i A^T (y - Ax^j), \quad (6)$$

for $i \in \{0, \dots, T-1\}$, where $x^0 = 0$. In other words, while calculating $\tilde{x}^i$, it uses not only the gradient calculated at the current step but also leverages all the gradients calculated from previous steps.

By using different choices for $\beta_0^i, \beta_1^i, \dots, \beta_i^i$ at each iteration, one can recover a large class of algorithms, including gradient descent, heavy-ball methods, and approximate message passing. As shown in Equation (6) and illustrated in Figure 2, we can rearrange the vector x^i and the gradients $\{A^T(y - Ax^j)\}_{j \in \{0, \dots, i\}}$ as images and view the expression as one-by-one convolutions over the images. Our simulation results reported later show that DeMUNs with trainable $\beta_0^i, \beta_1^i, \dots, \beta_i^i$ offer greater flexibility and better performance compared to fixed instances such as gradient descent or Nesterov's method.

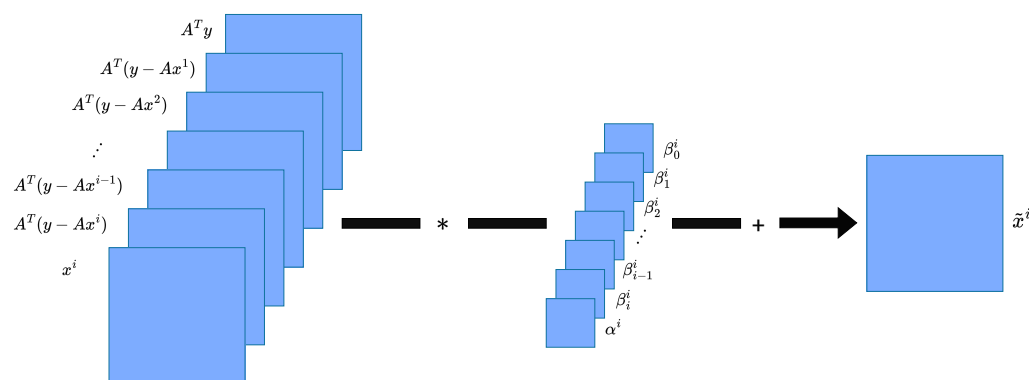


Figure 2. An example of the memory terms combined into a single image.

4. Our Four Main Hypotheses

4.1. Simulation Setup

Our goal in this section is to (1) show the effectiveness of the DeMUN by comparing its performance against different unrolled algorithms and (2) explore the impact of specific design choices. We conduct extensive ablation studies where we fix all but one design choice at each step and explore the performance of unrolled algorithms under different options for this choice. Based on these studies, we have developed several hypotheses aimed at simplifying the design of unrolled networks. We will outline these hypotheses and present simulation results that support them.

For all simulations below, we report results of four different sampling rates m/n for the measurement matrix A : 10%, 20%, 30%, and 40%. In Section 4, each entry in the measurement matrix is i.i.d. Gaussian, where $A_{ij} \sim \mathcal{N}(0, 1/m)$ for $A \in \mathbb{R}^{m \times n}$. While we will discuss the impact of the resolution on the performance of the algorithms, in the initial simulations, all training images have resolution 50×50 , and vectorizing the images leads to $n = 2500$. We primarily consider when the number of unrolled steps $T = 30$, with additional comparisons to performance at $T = 5, 15$, where illustrative. For all results below, we report the Peak Signal-to-Noise Ratio (PSNR) and the Structural Similarity Index Measure (SSIM) for the networks trained under the aforementioned sampling rates and number of projection steps on a test set of 2500 images. More details on data collection and processing, training of unrolled networks, and evaluation are deferred to Appendix A.

In our simulations, we adopt the general DnCNN architecture as outlined by Zhang et al. as our neural network projector P_C [52]. A DnCNN architecture with L intermediate layers consists of an input layer with 64 filters of size $3 \times 3 \times 1$ followed by an ReLU activation function to map the input image to 64 channels (It is of size $3 \times 3 \times 1$ since we assume the images are in grayscale.), L layers consisting of 64 filters of size $3 \times 3 \times 64$ followed by BatchNormalization and ReLU, and a final reconstruction layer with a single filter of size $3 \times 3 \times 64$ to map to the output dimension of $50 \times 50 \times 1$.

4.2. Overview of Our Simplifying Hypotheses

As previously described, we begin with four hypotheses, each of which contributes to improving the performance of unrolled networks, enhancing training practices, and simplifying the design process by reducing the number of decisions practitioners need to make. These hypotheses are based on extensive simulations and are reported below.

Hypothesis 1. *Unrolled networks trained with the loss function $\ell_{i,1}$ uniformly outperform their counterparts trained with ℓ_{Π} . Among the unrolling algorithms we tested, i.e., PGD, AMP, and Nesterov, DeMUNs offer superior recovery performance.*

Although we are primarily concerned with the quality of the final reconstruction $\|x^T - x^*\|_2^2$, we find that using the loss function $\ell_{i,1} = \sum_{i=1}^T \|x^i - x^*\|_2^2$ during training yields better recovery performance than focusing solely on the last layer. This improvement may be attributed to the smoother optimization landscape provided by using the intermediate loss, which guides the network more effectively towards better minima. We present our empirical evidence for suggesting this hypothesis in Section 4.3. With the advantage of using an unweighted intermediate loss function established, we next explore the impact of incorporating residual connections into unrolled networks.

Hypothesis 2. *DeMUNs trained using residual connections and loss function $\ell_{i,1}$ uniformly improve recovery performance compared to those trained without residual connections.*

Residual connections are known to alleviate issues such as vanishing gradients and facilitate the training of deeper networks by allowing gradients to propagate more effectively through the intermediate layers [53,54]. Specifically, we modify each projection step in our unrolled network to be $x^{i+1} = \tilde{x}^i + P_C(\tilde{x}^i)$. In verifying Hypothesis 2, we continue to use $\omega = 1$ (see the definition of intermediate loss in (4)). This ensures that any observed improvements can be directly attributed to the addition of residual connections rather than changes in the loss function. We present our empirical evidence for suggesting this hypothesis in Section 4.4. Having confirmed that both the use of an unweighted intermediate loss and the inclusion of residual connections improve recovery performance, we further investigate the sensitivity of our network to the specific shape of the loss function.

Hypothesis 3. *For training DeMUNs, there is no significant difference among the following loss functions: (1) $\ell_{i,1}$, (2) $\ell_{i,0.95}$, and (3) $\ell_{i,0.85}$. Furthermore, $\ell_{i,0.5}$, $\ell_{i,0.25}$, $\ell_{i,0.1}$, $\ell_{i,0.01}$, and $\ell_{s,5}$ perform worse than $\ell_{i,1}$.*

Hypothesis 4. *When we vary the number of layers, L , in the DnCNN from 5 to 15, the performance of DeMUNs remains largely unchanged, indicating that the number of layers has a negligible impact on its performance. However, increasing L from 3 to 5 provides a noticeable improvement in performance.*

Confirming these hypotheses provides a set of practical recommendations for designing unrolled networks that are both effective and robust across various settings.

4.3. Impact of Intermediate Loss

In this section, we aim to validate Hypothesis 1, which posits that deep unrolled networks trained with the unweighted intermediate loss function $\ell_{i,1}$ uniformly outperform their counterparts trained with the last-layer loss ℓ_{Π} . We consider the following algorithms:

1. Deep Memory Unrolled Network (DeMUN): Our proposed network that incorporates the memory of all the gradients into the unrolling process.

2. Projected Gradient Descent (PGD): The standard unrolled algorithm outlined in (2).
3. Nesterov's Accelerated First-Order Method (Nesterov): An optimization method that uses momentum to accelerate convergence [46].
4. Approximate Message Passing (AMP): An iterative algorithm tailored for linear inverse problems with Gaussian sensing matrices [6,15].

For all unrolled algorithms, we consider when all the projection steps are cast as direct projections of the form $x^{i+1} = P_C(\tilde{x}^i)$ and compare the performance between last-layer loss and unweighted intermediate loss. Figure 3 presents an example of a DnCNN architecture with $L = 3$ intermediate layers.

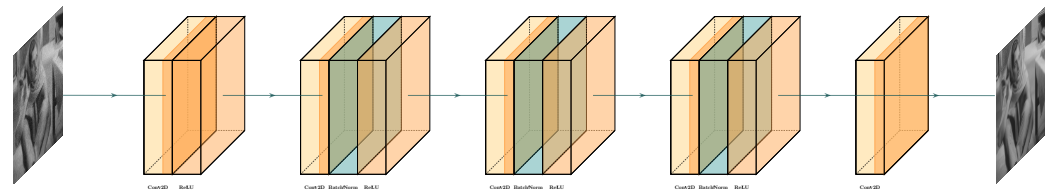


Figure 3. An example of the DnCNN architecture with $L = 3$ intermediate layers.

Regarding Table 1 and Figures 4 and 5, we make the following remarks.

- **Improved Performance with Intermediate Loss:** By analyzing the tables and graphs, we conclude that, across all four unrolled algorithms, training with the intermediate loss function $\ell_{i,1}$ consistently yields higher PSNR values compared to training with the last-layer loss, ℓ_{ll} .
- **Superiority of Deep Memory Unrolled Network:** Among all algorithms that we have unrolled, i.e., PGD, Nesterov, and AMP, the DeMUN achieves the highest PSNR values when trained with the intermediate loss, confirming our hypothesis. (However, we see that this is not always the case when using last-layer loss. A possible explanation is that our memory networks contain many parameters (especially with many projection steps) and may be stuck at a local minimum during training using the last-layer loss. In contrast, when adopting the intermediate loss function, the network needs to optimize its projection performance across all projection steps to minimize the loss. As a result, it may find better solutions, especially for the parameters that are involved in the earlier layers.) This is to be expected as DeMUNs encompass the other unrolled networks as special cases. During training, the data effectively determines which algorithm should be unrolled.

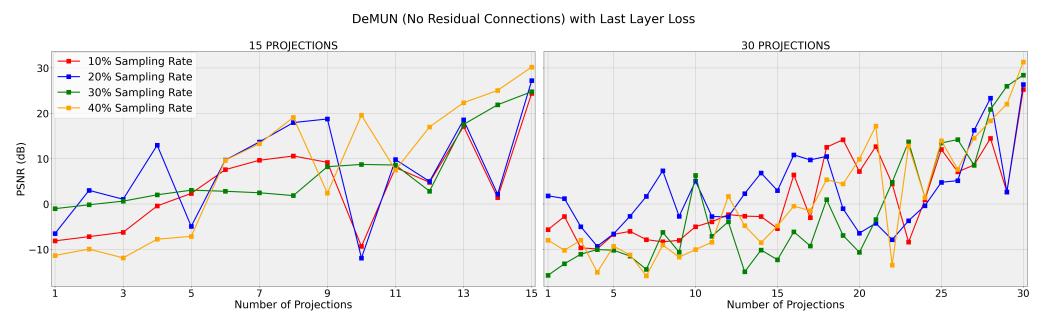
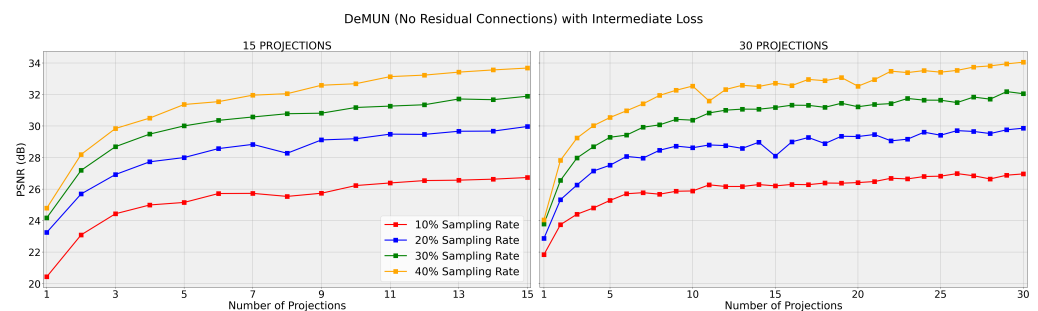
According to these observations, the intermediate loss may provide several benefits:

- **Avoiding Poor Local Minima:** Focusing solely on the output of the final layer may lead the network to suboptimal solutions (due to non-convexity). In comparison, the intermediate loss encourages the network to make meaningful progress at each step, which potentially reduces the risk of becoming stuck in poor local minima.
- **More Information during Backpropagation:** By including losses from all intermediate steps, the network receives more gradient information during autodifferentiation, which may be helpful in learning better representations and weights.

These empirical results strongly support our first hypothesis that incorporating information from all intermediate steps creates a more effective learning mechanism for the network.

Table 1. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different unrolled algorithms and loss functions ℓ_{II} and $\ell_{i,1}$.

Metric	m	Algorithm			
		DeMUN	PGD	Nesterov	AMP
ℓ_{II}					
PSNR	$0.1n$	25.27 ± 5.63	24.17 ± 5.61	24.48 ± 5.55	15.99 ± 2.99
	$0.2n$	26.37 ± 5.46	27.29 ± 5.96	27.00 ± 5.72	20.02 ± 2.52
	$0.3n$	28.44 ± 5.21	28.30 ± 5.41	29.24 ± 5.63	23.22 ± 2.82
	$0.4n$	31.32 ± 5.95	30.33 ± 5.44	29.95 ± 5.46	22.65 ± 2.72
SSIM	$0.1n$	0.643 ± 0.180	0.590 ± 0.181	0.611 ± 0.183	0.290 ± 0.122
	$0.2n$	0.697 ± 0.149	0.735 ± 0.144	0.731 ± 0.143	0.436 ± 0.155
	$0.3n$	0.793 ± 0.111	0.794 ± 0.110	0.817 ± 0.107	0.618 ± 0.109
	$0.4n$	0.879 ± 0.088	0.850 ± 0.085	0.837 ± 0.093	0.723 ± 0.094
$\ell_{i,1}$					
PSNR	$0.1n$	26.97 ± 6.42	26.51 ± 6.16	26.19 ± 5.78	26.72 ± 6.26
	$0.2n$	29.86 ± 6.55	29.06 ± 6.03	28.35 ± 5.66	29.12 ± 6.18
	$0.3n$	32.05 ± 6.54	30.87 ± 5.85	29.56 ± 5.21	31.24 ± 6.29
	$0.4n$	34.05 ± 6.71	32.33 ± 5.75	31.14 ± 5.26	32.87 ± 6.21
SSIM	$0.1n$	0.701 ± 0.180	0.686 ± 0.180	0.678 ± 0.182	0.693 ± 0.184
	$0.2n$	0.811 ± 0.135	0.796 ± 0.136	0.777 ± 0.140	0.790 ± 0.143
	$0.3n$	0.873 ± 0.101	0.855 ± 0.104	0.833 ± 0.106	0.855 ± 0.108
	$0.4n$	0.910 ± 0.075	0.892 ± 0.079	0.874 ± 0.084	0.894 ± 0.083

**Figure 4.** DeMUN (no residual connections) with loss ℓ_{II} . The networks are trained for $T = 15$ (left) and $T = 30$ (right), and the graph displays the PSNR after each intermediate projection. Both graphs share the same y-axis scale.**Figure 5.** DeMUN (no residual connections) with loss $\ell_{i,1}$. The networks are trained for $T = 15$ (left) and $T = 30$ (right), and the graph displays the PSNR after each intermediate projection. Both graphs share the same y-axis scale.

4.4. Impact of Residual Connections

Having verified that training with the intermediate loss function $\ell_{i,1}$ improves the recovery performance of unrolled networks, we now examine the effect of incorporating

residual connections of the form $x^{i+1} = \tilde{x}^i + P_C(\tilde{x}^i)$ into unrolled networks, as stated in Hypothesis 2, when fixing the choice of the unweighted intermediate loss function $\ell_{i,1}$. For comparison, in addition to the Deep Memory Unrolled Network, we include the results for unrolled networks based on PGD under the same conditions.

From Table 2 and Figure 6, we observe the following:

1. **Consistent Performance Improvement:** Including residual connections consistently improves the PSNR across all sampling rates and number of projection steps for both the deep memory- and PGD-based unrolled networks.
2. **Superior Performance of Deep Memory Network:** While both networks benefit from residual connections, the Deep Memory Unrolled Networks maintain superior performance over projected gradient descent in all scenarios.

These empirical results strongly support Hypothesis 2 that incorporating residual connections into the Deep Memory Unrolled Network further improves its performance on top of training with the unweighted intermediate loss function. The consistent improvement across different sampling rates and projection steps potentially highlights the value of residual connections in unrolled network architectures.

Table 2. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across residual and no residual connections.

Metric	m	DeMUN		PGD	
		No Residual	Residual	No Residual	Residual
PSNR	$0.1n$	26.97 ± 6.42	27.44 ± 6.88	26.51 ± 6.16	26.61 ± 6.77
	$0.2n$	29.86 ± 6.55	30.74 ± 7.32	29.06 ± 6.03	30.06 ± 6.97
	$0.3n$	32.05 ± 6.54	32.77 ± 7.09	30.87 ± 5.85	31.88 ± 6.84
	$0.4n$	34.05 ± 6.71	34.86 ± 7.30	32.33 ± 5.75	33.74 ± 6.81
SSIM	$0.1n$	0.701 ± 0.180	0.713 ± 0.181	0.686 ± 0.180	0.691 ± 0.178
	$0.2n$	0.811 ± 0.135	0.824 ± 0.133	0.796 ± 0.136	0.810 ± 0.139
	$0.3n$	0.873 ± 0.101	0.878 ± 0.102	0.855 ± 0.104	0.857 ± 0.127
	$0.4n$	0.910 ± 0.075	0.915 ± 0.075	0.892 ± 0.079	0.902 ± 0.080

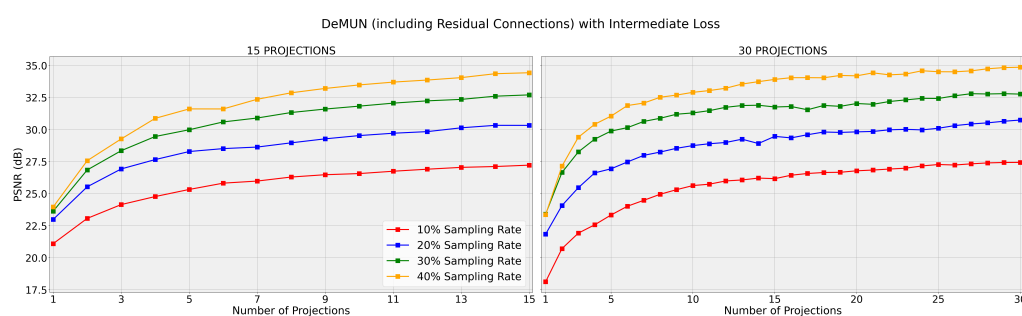


Figure 6. DeMUN (including residual connections) with loss $\ell_{i,1}$. The networks are trained for $T = 15$ (left) and $T = 30$ (right), and the graph displays the PSNR after each intermediate projection. Both graphs share the same y-axis scale.

4.5. Sensitivity to Other Loss Functions

Having identified that using an unweighted intermediate loss function and incorporating residual connections in Deep Memory Unrolled Networks offer superior performance, we now explore the sensitivity of our network to variations in the loss function as raised in Hypothesis 3. Specifically, we want to see whether different weighting schemes in the intermediate loss function or using a skip- L layer loss significantly impact the recovery performance. We consider the following variations of the loss function: $\ell_{i,\omega}$, where

$\omega \in \{0.95, 0.85, 0.75, 0.5, 0.25, 0.1, 0.01\}$ and $\ell_{s,5}$. We also include results for Deep Memory Unrolled Networks trained using ℓ_{II} with residual connections for comparison.

From the simulation results presented in Table 3, we are able to observe the following:

1. **Minimal Impact for $\omega \geq 0.75$:** When $\omega \in \{1, 0.95, 0.85, 0.75\}$, the recovery performance remains relatively consistent, with negligible differences in the PSNR values.
2. **Degradation with Small ω :** For $\omega \in \{0.5, 0.25, 0.1, 0.01\}$, there is a noticeable decrease in reconstruction quality. This decline may be attributed to the exponential down-weighting of the initial layers, which causes the network to focus excessively on the later iterations, potentially leading to suboptimal convergence.

We see that as long as the intermediate outputs receive sufficient emphasis during training, the network can output high-quality reconstructions. The decline in performance with smaller values of ω underscores the importance of adequately supervising the reconstruction of intermediate layers to guide the network toward the desirable recovery.

Table 3. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different loss functions.

Loss Function	Sampling Rate (m)			
	0.1n	0.2n	0.3n	0.4n
PSNR				
$\ell_{i,1}$	27.44 $\pm$ 6.88	30.74 $\pm$ 7.32	32.77 $\pm$ 7.09	34.86 $\pm$ 7.30
$\ell_{i,0.95}$	27.41 $\pm$ 6.78	30.63 $\pm$ 7.12	32.92 $\pm$ 7.23	34.34 $\pm$ 6.93
$\ell_{i,0.85}$	27.43 $\pm$ 6.95	30.47 $\pm$ 7.08	32.79 $\pm$ 7.20	34.79 $\pm$ 7.22
$\ell_{i,0.75}$	27.22 $\pm$ 6.61	30.62 $\pm$ 7.15	32.58 $\pm$ 6.92	34.55 $\pm$ 6.94
$\ell_{i,0.5}$	27.04 $\pm$ 6.58	30.06 $\pm$ 6.80	32.23 $\pm$ 6.74	33.73 $\pm$ 6.70
$\ell_{i,0.25}$	26.99 $\pm$ 6.62	29.92 $\pm$ 6.80	31.97 $\pm$ 6.66	33.43 $\pm$ 6.56
$\ell_{i,0.1}$	26.81 $\pm$ 6.67	29.28 $\pm$ 6.39	31.79 $\pm$ 6.72	33.04 $\pm$ 6.35
$\ell_{i,0.01}$	26.65 $\pm$ 6.31	29.63 $\pm$ 6.58	31.65 $\pm$ 6.64	33.50 $\pm$ 6.61
$\ell_{s,5}$	27.31 $\pm$ 6.61	30.52 $\pm$ 6.98	32.72 $\pm$ 6.99	34.71 $\pm$ 7.07
ℓ_{II}	26.75 $\pm$ 6.45	23.10 $\pm$ 4.14	29.24 $\pm$ 5.45	33.69 $\pm$ 6.69
SSIM				
$\ell_{i,1}$	0.713 $\pm$ 0.181	0.824 $\pm$ 0.133	0.878 $\pm$ 0.102	0.915 $\pm$ 0.075
$\ell_{i,0.95}$	0.715 $\pm$ 0.182	0.825 $\pm$ 0.133	0.881 $\pm$ 0.100	0.902 $\pm$ 0.102
$\ell_{i,0.85}$	0.713 $\pm$ 0.180	0.821 $\pm$ 0.132	0.879 $\pm$ 0.099	0.916 $\pm$ 0.075
$\ell_{i,0.75}$	0.707 $\pm$ 0.180	0.824 $\pm$ 0.134	0.877 $\pm$ 0.099	0.913 $\pm$ 0.076
$\ell_{i,0.5}$	0.703 $\pm$ 0.178	0.811 $\pm$ 0.138	0.873 $\pm$ 0.100	0.901 $\pm$ 0.086
$\ell_{i,0.25}$	0.697 $\pm$ 0.180	0.806 $\pm$ 0.135	0.870 $\pm$ 0.101	0.902 $\pm$ 0.080
$\ell_{i,0.1}$	0.692 $\pm$ 0.183	0.800 $\pm$ 0.134	0.866 $\pm$ 0.105	0.901 $\pm$ 0.082
$\ell_{i,0.01}$	0.690 $\pm$ 0.183	0.804 $\pm$ 0.136	0.861 $\pm$ 0.105	0.903 $\pm$ 0.079
$\ell_{s,5}$	0.712 $\pm$ 0.184	0.823 $\pm$ 0.134	0.878 $\pm$ 0.099	0.914 $\pm$ 0.079
ℓ_{II}	0.694 $\pm$ 0.180	0.546 $\pm$ 0.123	0.809 $\pm$ 0.105	0.904 $\pm$ 0.076

4.6. Impact of the Complexity of the Projection Step

In this section, we examine Hypothesis 4 by changing the number of intermediate layers L of the DnCNN architecture. We assume that there is no additive measurement noise and consider $L = 3, 5, 10$, and 15 layers. Our results are shown below.

We summarize our conclusions from Tables 4–6 below:

1. Increasing the number of layers from 5 to 15 results in negligible changes in the performance of DeMUNs regardless of the number of projection steps. By comparing Tables 4–6, we observe that the number of projections has a significantly greater impact on performance than the number of layers within each projection.

2. By comparing $L = 3$ and $L = 5$, we conclude that reducing the depth too drastically ($L \leq 3$) may impair the network's ability to learn complex features as convolutional neural networks rely on multiple layers to capture hierarchical representations [55].

We acknowledge that these conclusions may not necessarily extend to other projector architectures that do not rely on deep convolutional layers. Nevertheless, we believe this observation generalizes to other types of architectures when their capacity diminishes beyond a certain threshold, although we defer further investigation to future work. We address extensions to other types of measurement matrices in Section 5.3.

Table 4. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 5 projection steps across different network depths.

Metric	m	Network Depth (L)			
		$L = 3$	$L = 5$	$L = 10$	$L = 15$
PSNR	$0.1n$	26.33 ± 6.62	26.30 ± 6.48	26.28 ± 6.64	25.99 ± 6.31
	$0.2n$	29.03 ± 6.69	29.17 ± 6.74	29.27 ± 6.78	28.95 ± 6.44
	$0.3n$	30.72 ± 6.50	30.95 ± 6.58	31.15 ± 6.74	30.88 ± 6.43
	$0.4n$	32.22 ± 6.49	32.52 ± 6.65	32.67 ± 6.65	32.49 ± 6.30
SSIM	$0.1n$	0.673 ± 0.179	0.666 ± 0.182	0.674 ± 0.176	0.665 ± 0.176
	$0.2n$	0.782 ± 0.135	0.788 ± 0.135	0.793 ± 0.134	0.789 ± 0.133
	$0.3n$	0.842 ± 0.106	0.847 ± 0.105	0.852 ± 0.103	0.851 ± 0.098
	$0.4n$	0.881 ± 0.083	0.885 ± 0.080	0.889 ± 0.078	0.888 ± 0.079

Table 5. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 15 projection steps across different network depths.

Metric	m	Network Depth (L)			
		$L = 3$	$L = 5$	$L = 10$	$L = 15$
PSNR	$0.1n$	27.15 ± 6.87	27.22 ± 6.77	27.28 ± 6.85	27.38 ± 6.94
	$0.2n$	30.06 ± 6.94	30.33 ± 7.11	30.34 ± 6.94	30.19 ± 6.63
	$0.3n$	32.39 ± 7.09	32.70 ± 7.28	32.67 ± 7.20	32.63 ± 7.00
	$0.4n$	34.49 ± 7.22	34.43 ± 7.11	34.44 ± 7.16	34.29 ± 6.80
SSIM	$0.1n$	0.701 ± 0.182	0.708 ± 0.181	0.711 ± 0.179	0.710 ± 0.180
	$0.2n$	0.807 ± 0.139	0.811 ± 0.142	0.820 ± 0.131	0.818 ± 0.129
	$0.3n$	0.872 ± 0.102	0.874 ± 0.104	0.878 ± 0.099	0.880 ± 0.098
	$0.4n$	0.912 ± 0.076	0.908 ± 0.081	0.911 ± 0.079	0.914 ± 0.073

Table 6. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different network depths.

Metric	m	Network Depth (L)			
		$L = 3$	$L = 5$	$L = 10$	$L = 15$
PSNR	$0.1n$	27.43 ± 7.00	27.44 ± 6.88	27.51 ± 6.87	27.39 ± 6.99
	$0.2n$	30.32 ± 6.97	30.74 ± 7.32	30.70 ± 7.06	30.61 ± 7.05
	$0.3n$	32.67 ± 7.17	32.77 ± 7.09	32.87 ± 7.10	32.74 ± 6.97
	$0.4n$	34.44 ± 6.99	34.86 ± 7.30	34.70 ± 7.14	34.95 ± 7.34
SSIM	$0.1n$	0.710 ± 0.180	0.713 ± 0.181	0.714 ± 0.182	0.712 ± 0.182
	$0.2n$	0.812 ± 0.138	0.824 ± 0.133	0.829 ± 0.132	0.827 ± 0.132
	$0.3n$	0.874 ± 0.105	0.878 ± 0.102	0.882 ± 0.099	0.882 ± 0.099
	$0.4n$	0.912 ± 0.075	0.915 ± 0.075	0.913 ± 0.078	0.918 ± 0.075

5. Robustness of DeMUNs

In Section 4, we established, through extensive simulations, the superior performance of DeMUNs trained with unweighted intermediate loss $\ell_{i,1}$ and residual connections. The aim of this section is to assess the robustness of this configuration under various conditions. Specifically, we examine our network's performance under changes in the measurement matrix, the presence of additive noise, variations in input image resolution, and changes in projector capacity. These aspects represent the primary variables that practitioners must consider when deploying unrolled networks in real-world scenarios. Our extensive experiments demonstrate the adequacy and generalizability of our design choices. In the simulations presented in the following sections, we fix the image resolution to 50×50 when the resolution has not been specified.

5.1. Robustness to the Sampling Matrix

We first investigate our network's performance under different sampling matrix structures. In addition to the Gaussian random matrix used previously, we consider a Discrete Cosine Transform (DCT) matrix of the form $A = SF \in \mathbb{R}^{m \times n}$, where $S \in \mathbb{R}^{m \times n}$ is an undersampling matrix and F represents the 2D-DCT. We set the number of hidden layers for each projector (DnCNN) $L = 5$. Additional implementation details can be found in Appendix A. There are a few points that we would like to clarify here:

1. Table 7 demonstrates that our network maintains good performance when considering DCT-type measurement matrices as well. The network effectively adapts to the DCT matrices, achieving comparable or better PSNR values than the Gaussian forward model. This suggests that our design choices made based on our simulations on Gaussian forward models offer good performance for other types of matrices as well.
2. The performance improvement DeMUNs gain from additional projection steps on DCT forward models is typically less than the improvement achieved with additional projections on Gaussian matrices. Since there are no signs of overfitting concerning recovery performance, we believe that the user does not need to worry about the number of projection steps when designing the network.

Table 7. Average Test PSNR (dB) and SSIM $\pm$ standard deviation across different projection steps and sampling matrices.

m	Matrix	Projection Steps		
		5 Steps	15 Steps	30 Steps
		PSNR		
0.1n	Gaussian	26.30 $\pm$ 6.48	27.22 $\pm$ 6.77	27.44 $\pm$ 6.88
	DCT	28.42 $\pm$ 7.23	28.47 $\pm$ 7.19	28.52 $\pm$ 7.23
0.2n	Gaussian	29.17 $\pm$ 6.74	30.33 $\pm$ 7.11	30.74 $\pm$ 7.32
	DCT	30.00 $\pm$ 7.23	30.37 $\pm$ 7.37	30.48 $\pm$ 7.48
0.3n	Gaussian	30.95 $\pm$ 6.58	32.70 $\pm$ 7.28	32.77 $\pm$ 7.09
	DCT	31.50 $\pm$ 7.11	32.14 $\pm$ 7.49	32.20 $\pm$ 7.47
0.4n	Gaussian	32.52 $\pm$ 6.65	34.43 $\pm$ 7.11	34.86 $\pm$ 7.30
	DCT	33.22 $\pm$ 7.02	33.90 $\pm$ 7.44	34.04 $\pm$ 7.44

5.2. Robustness to Additive Noise

Next, we introduce additive noise and obtain measurements of the form $y = Ax + w$, where $w \sim \mathcal{N}(0, \sigma^2 I)$. We want to see if our design choices still offer good performance in the presence of additive noise. The primary objective of this section is to demonstrate that

the PSNR of DeMUN reconstructions gradually decreases as the noise level increases and overfitting does not occur as the number of projections increases.

We summarize some of our conclusions from Tables 8 and 9 below:

1. Despite additive measurement noise predictably lowering the recovery PSNR, its impact on performance is relatively controlled. In particular, as the noise level increases, the PSNR degrades at a rate significantly slower than the decrease in the input SNR. This suggests that the network effectively suppresses the measurement noise.
2. As the noise level grows, the marginal benefit of additional projection steps diminishes. In other words, fewer projection steps often suffice to achieve comparable reconstruction quality. As mentioned before and is clear from Table 8, still increasing the number of projections does not hurt the reconstruction performance of the network. Hence, in scenarios where the noise level is not known, practitioners may choose a number that works well for the noiseless setting and use it for the noisy settings as well.

Table 8. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different sampling matrices and noise levels.

Matrix	m	Noise Level (σ)			
		0.01	0.025	0.05	0.10
PSNR					
Gaussian	$0.1n$	27.08 ± 6.46	26.17 ± 5.74	24.89 ± 5.05	23.19 ± 4.54
	$0.2n$	29.63 ± 6.18	28.50 ± 5.57	26.71 ± 4.89	24.74 ± 4.56
	$0.3n$	31.58 ± 6.09	29.71 ± 5.15	27.78 ± 4.69	25.60 ± 4.45
	$0.4n$	32.82 ± 5.76	30.70 ± 4.88	28.63 ± 4.53	26.27 ± 4.43
DCT	$0.1n$	28.34 ± 6.91	27.97 ± 6.50	27.36 ± 5.95	26.35 ± 5.35
	$0.2n$	30.05 ± 6.87	29.16 ± 6.14	28.21 ± 5.72	26.73 ± 5.14
	$0.3n$	31.44 ± 6.58	30.31 ± 5.89	28.91 ± 5.42	27.14 ± 5.00
	$0.4n$	32.91 ± 6.27	31.32 ± 5.49	29.58 ± 5.02	27.56 ± 4.84
SSIM					
Gaussian	$0.1n$	0.703 ± 0.180	0.679 ± 0.172	0.636 ± 0.171	0.567 ± 0.180
	$0.2n$	0.803 ± 0.136	0.782 ± 0.131	0.722 ± 0.139	0.642 ± 0.159
	$0.3n$	0.865 ± 0.099	0.825 ± 0.103	0.768 ± 0.121	0.684 ± 0.144
	$0.4n$	0.893 ± 0.077	0.858 ± 0.083	0.802 ± 0.102	0.719 ± 0.130
DCT	$0.1n$	0.781 ± 0.156	0.771 ± 0.155	0.752 ± 0.154	0.710 ± 0.158
	$0.2n$	0.825 ± 0.130	0.809 ± 0.129	0.780 ± 0.134	0.727 ± 0.145
	$0.3n$	0.863 ± 0.104	0.842 ± 0.105	0.806 ± 0.114	0.745 ± 0.132
	$0.4n$	0.894 ± 0.081	0.871 ± 0.083	0.830 ± 0.094	0.764 ± 0.122

Table 9. Test input SNR (dB) under different sampling rates and noise levels.

m	Matrix	$\sigma = 0.01$	$\sigma = 0.025$	$\sigma = 0.05$	$\sigma = 0.10$
0.1n	Gaussian	32.19	24.23	18.21	12.19
	DCT	42.61	34.65	28.63	22.61
0.2n	Gaussian	32.55	24.59	18.57	12.55
	DCT	39.61	31.65	25.63	19.61
0.3n	Gaussian	32.57	24.62	18.59	12.57
	DCT	37.87	29.91	23.89	17.87
0.4n	Gaussian	32.49	24.53	18.51	12.49
	DCT	36.63	28.67	22.65	16.63

5.3. Robustness of Hypothesis 4 to Sampling Matrix and Additive Noise

The main goal of this section is to evaluate the robustness of Hypothesis 4 in response to changes in the measurement matrix and measurement noise. We first assume that there is no additive noise and consider $L = 3, 5, 10$, and 15 layers. We then evaluate the performance of DeMUNs on DCT-type matrices described in Section 5.1.

As evident from Table 10, increasing L from 5 to 15 does not provide a noticeable improvement for DCT-type matrices. One could also argue that, in most cases for DCT-type matrices, the performance gain from increasing L from 3 to 5 is marginal.

Next, we study the accuracy of Hypothesis 4 when additive noise is present in the measurements. Here, we consider three noise levels $\sigma \in \{0.01, 0.025, 0.05\}$ and test depths of $L = 3, 5$, and 10. The results are presented in Tables 11 and 12.

Table 10. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different networks depths with DCT matrices.

m	Network Depth (L)			
	$L = 3$	$L = 5$	$L = 10$	$L = 15$
PSNR				
$0.1n$	28.51 ± 7.29	28.52 ± 7.23	28.52 ± 7.22	28.51 ± 7.17
$0.2n$	30.34 ± 7.39	30.48 ± 7.48	30.52 ± 7.46	30.36 ± 7.35
$0.3n$	32.18 ± 7.61	32.20 ± 7.47	32.18 ± 7.40	31.98 ± 7.29
$0.4n$	33.88 ± 7.44	34.04 ± 7.44	34.04 ± 7.44	33.91 ± 7.21
SSIM				
$0.1n$	0.783 ± 0.157	0.785 ± 0.157	0.785 ± 0.157	0.785 ± 0.157
$0.2n$	0.829 ± 0.132	0.831 ± 0.131	0.832 ± 0.131	0.830 ± 0.131
$0.3n$	0.869 ± 0.107	0.869 ± 0.105	0.871 ± 0.105	0.868 ± 0.104
$0.4n$	0.903 ± 0.083	0.903 ± 0.082	0.906 ± 0.081	0.906 ± 0.081

Table 11. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different networks depths and noise levels with Gaussian matrices.

Metric	σ	L	Sampling Rate (m)			
			$0.1n$	$0.2n$	$0.3n$	$0.4n$
PSNR	0.01	3	26.95 ± 6.50	29.54 ± 6.18	31.42 ± 6.04	32.81 ± 5.75
		5	27.08 ± 6.46	29.63 ± 6.18	31.58 ± 6.09	32.82 ± 5.76
		10	27.03 ± 6.38	30.06 ± 6.50	31.82 ± 6.27	33.06 ± 5.89
	0.025	3	26.03 ± 5.68	28.30 ± 5.47	29.61 ± 5.16	30.70 ± 4.94
		5	26.17 ± 5.74	28.50 ± 5.57	29.71 ± 5.15	30.70 ± 4.88
		10	26.19 ± 5.65	28.59 ± 5.60	29.98 ± 5.45	30.93 ± 5.04
	0.05	3	24.81 ± 5.04	26.59 ± 4.89	27.83 ± 4.86	28.49 ± 4.49
		5	24.89 ± 5.05	26.71 ± 4.89	27.78 ± 4.69	28.63 ± 4.53
		10	25.03 ± 5.17	26.93 ± 5.13	28.11 ± 5.03	28.71 ± 4.67
SSIM	0.01	3	0.698 ± 0.178	0.805 ± 0.132	0.860 ± 0.099	0.895 ± 0.075
		5	0.703 ± 0.180	0.803 ± 0.136	0.865 ± 0.099	0.893 ± 0.077
		10	0.700 ± 0.180	0.818 ± 0.130	0.869 ± 0.098	0.899 ± 0.075
	0.025	3	0.677 ± 0.174	0.773 ± 0.135	0.823 ± 0.104	0.856 ± 0.086
		5	0.679 ± 0.172	0.782 ± 0.131	0.825 ± 0.103	0.858 ± 0.083
		10	0.686 ± 0.171	0.784 ± 0.131	0.833 ± 0.104	0.864 ± 0.083
	0.05	3	0.633 ± 0.171	0.718 ± 0.138	0.771 ± 0.119	0.798 ± 0.102
		5	0.636 ± 0.171	0.722 ± 0.139	0.768 ± 0.121	0.802 ± 0.102
		10	0.641 ± 0.174	0.728 ± 0.144	0.781 ± 0.119	0.804 ± 0.102

Table 12. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different network depths and noise levels with DCT matrices.

Metric	σ	L	Sampling Rate (m)			
			$0.1n$	$0.2n$	$0.3n$	$0.4n$
PSNR	0.01	3	28.32 $\pm$ 6.94	29.94 $\pm$ 6.84	31.42 $\pm$ 6.65	32.79 $\pm$ 6.33
		5	28.34 $\pm$ 6.91	30.05 $\pm$ 6.87	31.44 $\pm$ 6.58	32.91 $\pm$ 6.27
		10	28.38 $\pm$ 6.98	30.10 $\pm$ 6.94	31.53 $\pm$ 6.69	33.07 $\pm$ 6.51
	0.025	3	27.97 $\pm$ 6.52	29.08 $\pm$ 6.13	30.25 $\pm$ 5.94	31.26 $\pm$ 5.55
		5	27.97 $\pm$ 6.50	29.16 $\pm$ 6.14	30.31 $\pm$ 5.89	31.32 $\pm$ 5.49
		10	27.98 $\pm$ 6.53	29.32 $\pm$ 6.36	30.37 $\pm$ 5.98	31.45 $\pm$ 5.59
	0.05	3	27.34 $\pm$ 5.96	28.13 $\pm$ 5.68	28.88 $\pm$ 5.46	29.57 $\pm$ 5.13
		5	27.36 $\pm$ 5.95	28.21 $\pm$ 5.72	28.91 $\pm$ 5.42	29.58 $\pm$ 5.02
		10	27.37 $\pm$ 5.97	28.25 $\pm$ 5.77	28.91 $\pm$ 5.40	29.71 $\pm$ 5.17
SSIM	0.01	3	0.780 $\pm$ 0.157	0.823 $\pm$ 0.131	0.861 $\pm$ 0.106	0.893 $\pm$ 0.082
		5	0.781 $\pm$ 0.156	0.825 $\pm$ 0.130	0.863 $\pm$ 0.104	0.894 $\pm$ 0.081
		10	0.782 $\pm$ 0.156	0.827 $\pm$ 0.130	0.864 $\pm$ 0.103	0.897 $\pm$ 0.080
	0.025	3	0.770 $\pm$ 0.155	0.806 $\pm$ 0.130	0.840 $\pm$ 0.107	0.869 $\pm$ 0.086
		5	0.771 $\pm$ 0.155	0.809 $\pm$ 0.129	0.842 $\pm$ 0.105	0.871 $\pm$ 0.083
		10	0.772 $\pm$ 0.155	0.811 $\pm$ 0.129	0.844 $\pm$ 0.105	0.873 $\pm$ 0.084
	0.05	3	0.750 $\pm$ 0.154	0.779 $\pm$ 0.133	0.804 $\pm$ 0.116	0.827 $\pm$ 0.100
		5	0.752 $\pm$ 0.154	0.780 $\pm$ 0.134	0.806 $\pm$ 0.114	0.830 $\pm$ 0.094
		10	0.751 $\pm$ 0.154	0.783 $\pm$ 0.133	0.807 $\pm$ 0.112	0.832 $\pm$ 0.098

These results strongly suggest that, even in the presence of additive noise, increasing L does not offer substantial gain in the performance of DeMUNs. Given that the improvement in recovery performance is marginal when increasing the projector capacity, this suggests that simple architectures like DnCNN with very few convolutional layers may be sufficient for practical applications where measurement noise is present, offering potential computational savings without significant performance degradation.

5.4. Robustness to Image Resolution

Finally, we assess the DeMUN's performance across different image resolutions. We test resolutions of 32×32 , 50×50 , 64×64 , and 80×80 , fixing the measurement matrices and removing measurement noise. There are two main questions we aim to address here: (1) Do we need more or fewer projections as we increase the number of projections? (2) How should we set the number of layers L in the projection as we increase/decrease resolution? As before, we first set the number of intermediate layers of each projector to $L = 5$.

We observe from Table 13 that, as the image resolution increases, the network's recovery performance generally improves. This is possibly due to the presence of more information in higher-resolution images, which helps the network in learning more detailed structural properties.

Table 13. Average test PSNR (dB) and SSIM $\pm$ standard deviation for 30 projection steps across different image resolutions and sampling matrices.

Matrix	m	Image Size			
		32×32	50×50	64×64	80×80
PSNR					
Gaussian	$0.1n$	27.20 ± 7.38	27.44 ± 6.88	28.18 ± 7.21	28.31 ± 6.96
	$0.2n$	29.78 ± 7.08	30.74 ± 7.32	30.91 ± 6.92	31.54 ± 7.23
	$0.3n$	32.26 ± 7.44	32.77 ± 7.09	33.22 ± 7.21	33.86 ± 7.32
	$0.4n$	33.70 ± 7.21	34.86 ± 7.30	34.99 ± 6.91	35.55 ± 7.06
DCT	$0.1n$	28.70 ± 7.70	28.52 ± 7.23	28.64 ± 7.12	28.83 ± 7.09
	$0.2n$	30.49 ± 7.84	30.48 ± 7.48	30.85 ± 7.29	31.14 ± 7.42
	$0.3n$	31.61 ± 7.58	32.20 ± 7.47	32.57 ± 7.40	33.35 ± 7.75
	$0.4n$	33.46 ± 7.67	34.04 ± 7.44	34.36 ± 7.48	35.01 ± 7.69
SSIM					
Gaussian	$0.1n$	0.699 ± 0.193	0.713 ± 0.181	0.728 ± 0.181	0.734 ± 0.176
	$0.2n$	0.806 ± 0.140	0.824 ± 0.133	0.829 ± 0.129	0.831 ± 0.134
	$0.3n$	0.869 ± 0.108	0.878 ± 0.102	0.878 ± 0.107	0.891 ± 0.095
	$0.4n$	0.902 ± 0.088	0.915 ± 0.075	0.918 ± 0.071	0.919 ± 0.072
DCT	$0.1n$	0.777 ± 0.166	0.785 ± 0.157	0.790 ± 0.154	0.796 ± 0.151
	$0.2n$	0.826 ± 0.138	0.831 ± 0.131	0.830 ± 0.133	0.841 ± 0.124
	$0.3n$	0.858 ± 0.115	0.869 ± 0.105	0.872 ± 0.104	0.885 ± 0.099
	$0.4n$	0.897 ± 0.088	0.903 ± 0.082	0.908 ± 0.079	0.914 ± 0.077

6. Conclusions

In this paper, we conducted a comprehensive empirical study on the design choices for unrolled networks in solving linear inverse problems. As our first step, we introduced the Deep Memory Unrolled Network (DeMUN), which leverages the history of all gradients and generalizes a wide range of existing unrolled networks. This approach was designed to (1) allow the data to decide on the optimal choice of algorithm to be unrolled and (2) improve recovery performance. A byproduct of our choice is that users do not need to decide which algorithm they need to unroll. Figure 7 presents examples of recovered images under DCT matrix with 30 projections across different sampling rates.

Through extensive simulations, we demonstrated that training the DeMUN with an unweighted intermediate loss function and incorporating residual connections represents the best existing practice (among the ones studied in this paper) for optimizing these networks. This approach delivers superior performance compared to existing unrolled algorithms, highlighting its effectiveness and versatility.

We also presented experiments that exhibit the robustness of our design choices to a wide range of conditions, including different measurement matrices, additive noise levels, and image resolutions. Hence, our results offer practical guidelines and rules of thumb for selecting the loss function for training, structuring the unrolled network, determining the required number of projections, and deciding on the appropriate number of layers. These insights simplify the design and optimization of such networks for a wide range of applications, and we expect them to serve as a useful reference for researchers and practitioners in designing effective unrolled networks for linear inverse problems across various settings.

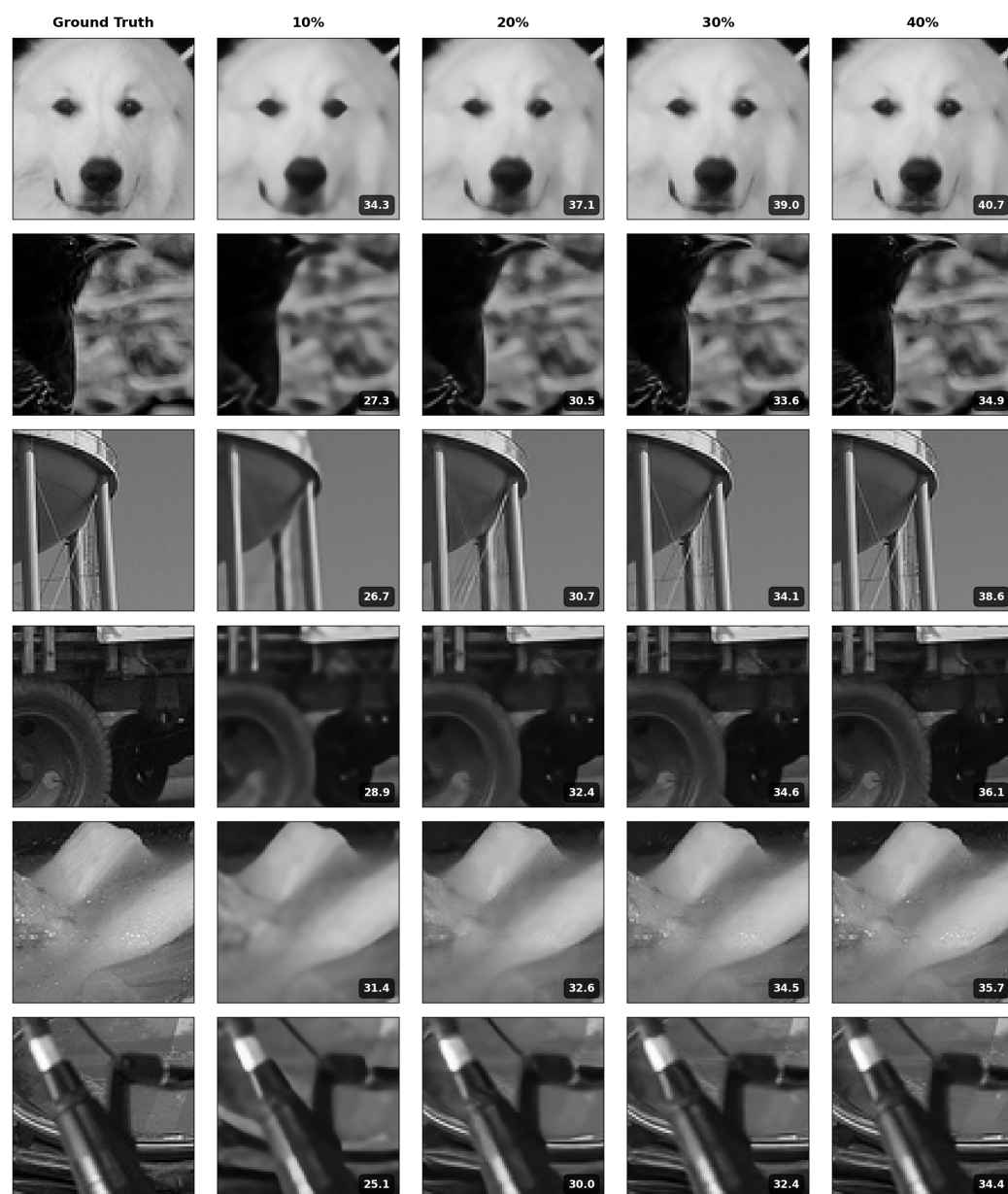


Figure 7. Examples of recovered images (80×80) under DCT matrix with 30 projections across different sampling rates. PSNR values shown in dB.

Author Contributions: Conceptualization, S.J. and A.M.; data curation, Y.C.; formal analysis, Y.C.; investigation, Y.C.; methodology, Y.C. and A.M.; project administration, A.M.; software, Y.C.; supervision, X.C., S.J. and A.M.; validation, Y.C.; visualization, Y.C.; writing—original draft, Y.C., X.C. and A.M.; writing—review and editing, Y.C., X.C. and A.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The accompanying code can be found at <https://github.com/YuxiChen25/Memory-Net-Inverse> (accessed on 9 August 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Experimental Setup

Below, we discuss the implementation details abbreviated in the sections above. Our networks are trained with NVIDIA A100 SXM4, H100 PCIE, H100 SXM5, and GH200.

Appendix A.1. Implementation Details

Our dataset is generated from the 50K validation images from the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC2012). (<https://www.image-net.org/challenges/LSVRC/2012/>, accessed on 9 August 2025). For each image from ILSVRC2012, we first convert it to grayscale and then crop the center $3k \times 3k$ region, where $k = 32, 50, 64$, or 80 depending on the image resolution. Then, each cropped image is converted into 9 images of size $k \times k$, where each smaller image is individually converted into a length $n = k^2$ vector $\tilde{x}$.

To generate the observation matrix $A \in \mathbb{R}^{m \times n}$ for Gaussian matrices, we draw $A_{ij} \sim \mathcal{N}(0, 1/m)$. For DCT matrices, the measurement matrix takes the form $A = SF$, where $S \in \mathbb{R}^{m \times n}$ is an undersampling matrix and $F \in \mathbb{R}^{n \times n}$ is the 2D-Discrete Cosine Transform matrix generated by the Kronecker product of two 1D-DCT matrices of size $k \times k$. To generate the subsampling matrix S , we adopt the following policy: for 10% sampling rate, we always sample the 10% of the 2D-DCT transformed image located approximately in the top-left corner by fixing the indices in advance. For each increased sampling rate, we randomly sample indices located elsewhere in the transformed image. For both settings, we normalize the measurement matrix as outlined in Section A.2.2 to obtain A' . Then, each observation is generated through the process $y = A'x + w$, where x is the normalized vector $x = \tilde{x}/255$. This generates 450K compressed measurements $y \in \mathbb{R}^n$ along with their ground-truth values x .

Appendix A.2. Training and Evaluation Details

Appendix A.2.1. Training and Test Data

To train and test the network, we take the first 25K images from the processed dataset and allot 2500 images for testing and 22500 for training. The training set is further partitioned into 18K images for training the network and 4500 images for validation. For training unrolled networks, we use a batch size of 32 and set the number of training epochs to 300. The learning rate is set to 1×10^{-4} using the ADAM Optimizer without any regularization. We select the model weights corresponding to the epoch that has the lowest validation loss evaluated on the 4500 images using the mean-squared-error. The remaining 2500 images are used to report the test PSNR above.

Appendix A.2.2. Initialization

In the case of Deep Memory Unrolled Networks (DeMUNs), the trainable weights can be partitioned into two categories: (1) the weights of neural networks that are used in the projector units of Figure 1, and (2) the weights α^i and $\{\beta_j^i\}$ in the 1×1 convolution that are used in the gradient steps. We adopt the following approach to initialize the weights:

1. We calculate the maximum ℓ^2 row norm of the matrix: $\|A\|_{\infty,2} = \max_{1 \leq i \leq m} \sqrt{\sum_{j=1}^n a_{ij}^2}$. Subsequently, we normalize A according to $A' = A/\|A\|_{\infty,2}$ to obtain our sampling matrix.
2. For each gradient step i , we initialize the first two terms α^i, β_i^1 to 1 and set all other weights to 0.

3. All other network projector weights $\{P_C^i\}_t$ are initialized randomly [12,17,48]. For instance, in PyTorch v2.8.0, convolutional weights are initialized by sampling from $\text{Uniform}(-\sqrt{k}, \sqrt{k})$, where

$$k = \frac{\text{groups}}{C_{in} \cdot \prod_{i=0}^1 \text{kernel size}_i}$$

Here, *groups* specifies independent channel groups (defaulting to 1), C_{in} is the number of input channels, and kernel size_i represents the kernel size along the i -th dimension (<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>, accessed on 9 August 2025).

The normalization applied in Step 1 ensures that our training process is robust to the scaling of the measurement matrix (or forward operator) A . Note that multiplying the measurement matrix by a factor, such as 0.001 or 1000, does not affect the inherent complexity of the problem. Hence, we expect our recovery method to produce the same estimate. However, in neural network models, where numerous multiplications and additions occur, extremely large or small values can lead to numerical issues. Additionally, due to the non-convexity of the training error, an initialization that performs well at one scale may not perform as effectively at another scale of the measurement matrix since it may lead to a different local minimum. The normalization introduced in Step 1 is designed to address these challenges, ensuring that both the initialization and the performance of the learned models remain robust to the scaling of A .

For Step 2, we note that the first two memory terms α^i and β_i^i correspond to x^i and $A^T(y - Ax^i)$. Initializing these terms with 1 recovers the standard projected gradient descent step $\tilde{x}^i = x^i + A^T(y - Ax^i)$ with unit step size to be fed into the neural network projector. Therefore, this scheme can be seen as initializing the memory terms to be standard projected gradient descent and making the weights corresponding to terms $\{\beta_j^i\}_{j \in \{0, \dots, i-1\}}$ in the memory to be learned as the training process progresses.

Appendix A.3. Unrolled Algorithms

Here, we provide supplementary details on the unrolled networks based on Nesterov's first-order method and approximate message passing used as comparison methods in Section 4.3.

Appendix A.3.1. Unrolled Nesterov's First-Order Method

For Nesterov's first-order method, each iteration proceeds as follows starting from $i = 0$:

$$\begin{aligned}\tilde{x}^i &= x_n^i + \mu A^T(y - Ax_n^i) \\ x^{i+1} &= P_C(\tilde{x}^i) \\ x_n^{i+1} &= x^{i+1} + \left(\frac{t_{i+1} - 1}{t_{i+2}}\right)(x^{i+1} - x^i)\end{aligned}$$

Here, $x_n^0 = x^0 = 0$ and $t_{i+1} = \frac{1 + \sqrt{1 + 4t_i^2}}{2}$, with $t_1 = 1$. The weights multiplying the linear combination are fixed and not backpropagated during the optimization process.

Appendix A.3.2. Unrolled Approximate Message Passing

For unrolled approximate message passing, each iteration proceeds in the following way:

$$\tilde{x}^i = x^i + A^T z^i$$

$$x^{i+1} = P_C(\tilde{x}^i)$$

$$z^{i+1} = y - Ax^{i+1} + z^i \operatorname{div}(P_C(x^i + A^T z^i))/m$$

Here, $z^0 = y$, and $\operatorname{div}(\cdot)$ is an estimate of the divergence of the projector estimated using a single Monte Carlo sample [6].

References

1. Donoho, D. Compressed sensing. *IEEE Trans. Inf. Theory* **2006**, *52*, 1289–1306. [\[CrossRef\]](#)
2. Lustig, M.; Donoho, D.; Pauly, J.M. Sparse MRI: The application of compressed sensing for rapid MR imaging. *Magn. Reson. Med.* **2007**, *58*, 1182–1195. [\[CrossRef\]](#)
3. Candes, E.; Wakin, M. An Introduction To Compressive Sampling. *IEEE Signal Process. Mag.* **2008**, *25*, 21–30. [\[CrossRef\]](#)
4. Baraniuk, R.G. Compressive sensing [lecture notes]. *IEEE Signal Process. Mag.* **2007**, *24*, 118–121. [\[CrossRef\]](#)
5. Donoho, D.L.; Maleki, A.; Montanari, A. Message-passing algorithms for compressed sensing. *Proc. Natl. Acad. Sci. USA* **2009**, *106*, 18914–18919. [\[CrossRef\]](#)
6. Metzler, C.A.; Maleki, A.; Baraniuk, R.G. From denoising to compressed sensing. *IEEE Trans. Inf. Theory* **2016**, *62*, 5117–5144. [\[CrossRef\]](#)
7. Venkatakrishnan, S.V.; Bouman, C.A.; Wohlberg, B. Plug-and-Play priors for model based reconstruction. In Proceedings of the 2013 IEEE Global Conference on Signal and Information Processing, Austin, TX, USA, 3–5 December 2013; pp. 945–948. [\[CrossRef\]](#)
8. Jalali, S.; Maleki, A. From compression to compressed sensing. *Appl. Comput. Harmon. Anal.* **2016**, *40*, 352–385. [\[CrossRef\]](#)
9. Beygi, S.; Jalali, S.; Maleki, A.; Mitra, U. An efficient algorithm for compression-based compressed sensing. *Inf. Inference J. IMA* **2019**, *8*, 343–375. [\[CrossRef\]](#)
10. Romano, Y.; Elad, M.; Milanfar, P. The Little Engine that Could: Regularization by Denoising (RED). *arXiv* **2017**, arXiv:1611.02862. [\[CrossRef\]](#)
11. Yang, Y.; Sun, J.; Li, H.; Xu, Z. Deep ADMM-Net for Compressive Sensing MRI. In *Advances in Neural Information Processing Systems*; Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., Garnett, R., Eds.; Curran Associates, Inc.: Red Hook, NY, USA, 2016; Volume 29.
12. Mousavi, A.; Baraniuk, R.G. Learning to invert: Signal recovery via Deep Convolutional Networks. In Proceedings of the 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), New Orleans, LA, USA, 5–9 March 2017; pp. 2272–2276. [\[CrossRef\]](#)
13. Chang, J.R.; Li, C.L.; Poczos, B.; Vijaya Kumar, B.; Sankaranarayanan, A.C. One Network to Solve Them All—Solving Linear Inverse Problems Using Deep Projection Models. In Proceedings of the 2017 IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 22–29 October 2017; pp. 5889–5898. [\[CrossRef\]](#)
14. Mousavi, A.; Dasarthy, G.; Baraniuk, R.G. DeepCodec: Adaptive Sensing and Recovery via Deep Convolutional Neural Networks. *arXiv* **2017**, arXiv:1707.03386. [\[CrossRef\]](#)
15. Metzler, C.A.; Mousavi, A.; Baraniuk, R.G. Learned D-AMP: Principled Neural Network based Compressive Image Recovery. *arXiv* **2017**, arXiv:1704.06625. [\[CrossRef\]](#)
16. McCann, M.T.; Jin, K.H.; Unser, M. Convolutional Neural Networks for Inverse Problems in Imaging: A Review. *IEEE Signal Process. Mag.* **2017**, *34*, 85–95. [\[CrossRef\]](#)
17. Zhang, J.; Ghanem, B. ISTA-Net: Interpretable Optimization-Inspired Deep Network for Image Compressive Sensing. *arXiv* **2018**, arXiv:1706.07929.
18. Diamond, S.; Sitzmann, V.; Heide, F.; Wetzstein, G. Unrolled Optimization with Deep Priors. *arXiv* **2018**, arXiv:1705.08041. [\[CrossRef\]](#)
19. Schlemper, J.; Caballero, J.; Hajnal, J.V.; Price, A.N.; Rueckert, D. A Deep Cascade of Convolutional Neural Networks for Dynamic MR Image Reconstruction. *IEEE Trans. Med Imaging* **2018**, *37*, 491–503. [\[CrossRef\]](#)
20. Gilton, D.; Ongie, G.; Willett, R. Neumann Networks for Inverse Problems in Imaging. *arXiv* **2019**, arXiv:1901.03707. [\[CrossRef\]](#)
21. Aggarwal, H.K.; Mani, M.P.; Jacob, M. MoDL: Model-Based Deep Learning Architecture for Inverse Problems. *IEEE Trans. Med Imaging* **2019**, *38*, 394–405. [\[CrossRef\]](#)
22. Ongie, G.; Jalal, A.; Metzler, C.A.; Baraniuk, R.G.; Dimakis, A.G.; Willett, R. Deep Learning Techniques for Inverse Problems in Imaging. *IEEE J. Sel. Areas Inf. Theory* **2020**, *1*, 39–56. [\[CrossRef\]](#)

23. Veen, D.V.; Jalal, A.; Soltanolkotabi, M.; Price, E.; Vishwanath, S.; Dimakis, A.G. Compressed Sensing with Deep Image Prior and Learned Regularization. *arXiv* **2020**, arXiv:1806.06438. [\[CrossRef\]](#)
24. Gilton, D.; Ongie, G.; Willett, R. Deep Equilibrium Architectures for Inverse Problems in Imaging. *IEEE Trans. Comput. Imaging* **2021**, *7*, 1123–1133. [\[CrossRef\]](#)
25. Gilton, D.; Ongie, G.; Willett, R. Model Adaptation for Inverse Problems in Imaging. *IEEE Trans. Comput. Imaging* **2021**, *7*, 661–674. [\[CrossRef\]](#)
26. Kadkhodaie, Z.; Simoncelli, E. Stochastic Solutions for Linear Inverse Problems using the Prior Implicit in a Denoiser. In *Advances in Neural Information Processing Systems*; Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J.W., Eds.; Curran Associates, Inc.: Red Hook, NY, USA, 2021; Volume 34, pp. 13242–13254.
27. Shastri, S.K.; Ahmad, R.; Metzler, C.A.; Schniter, P. Denoising Generalized Expectation-Consistent Approximation for MR Image Recovery. *IEEE J. Sel. Areas Inf. Theory* **2022**, *3*, 528–542. [\[CrossRef\]](#) [\[PubMed\]](#)
28. Rout, L.; Chen, Y.; Kumar, A.; Caramanis, C.; Shakkottai, S.; Chu, W.S. Beyond First-Order Tweedie: Solving Inverse Problems using Latent Diffusion. *arXiv* **2023**, arXiv:2312.00852. [\[CrossRef\]](#)
29. Zhang, J.; Chen, B.; Xiong, R.; Zhang, Y. Physics-Inspired Compressive Sensing: Beyond deep unrolling. *IEEE Signal Process. Mag.* **2023**, *40*, 58–72. [\[CrossRef\]](#)
30. Kamilov, U.S.; Bouman, C.A.; Buzzard, G.T.; Wohlberg, B. Plug-and-play methods for integrating physical and learned models in computational imaging: Theory, algorithms, and applications. *IEEE Signal Process. Mag.* **2023**, *40*, 85–97. [\[CrossRef\]](#)
31. Gan, W.; Hu, Y.; Liu, J.; An, H.; Kamilov, U. Block coordinate plug-and-play methods for blind inverse problems. In Proceedings of the NIPS'23: 37th International Conference on Neural Information Processing Systems, New Orleans, LA, USA, 10–16 December 2023.
32. Gan, W.; Zhai, Q.; McCann, M.T.; Cardona, C.G.; Kamilov, U.S.; Wohlberg, B. PtychoDV: Vision Transformer-Based Deep Unrolling Network for Ptychographic Image Reconstruction. *IEEE Open J. Signal Process.* **2024**, *5*, 539–547. [\[CrossRef\]](#)
33. Hu, Y.; Peng, A.; Gan, W.; Milanfar, P.; Delbracio, M.; Kamilov, U.S. Stochastic Deep Restoration Priors for Imaging Inverse Problems. *arXiv* **2024**, arXiv:2410.02057. [\[CrossRef\]](#)
34. Chung, H.; Lee, S.; Ye, J.C. Decomposed Diffusion Sampler for Accelerating Large-Scale Inverse Problems. *arXiv* **2024**, arXiv:2303.05754. [\[CrossRef\]](#)
35. Chen, B.; Zhang, J. Practical Compact Deep Compressed Sensing. *arXiv* **2024**, arXiv:2411.13081. [\[CrossRef\]](#)
36. Chen, B.; Zhang, X.; Liu, S.; Zhang, Y.; Zhang, J. Self-Supervised Scalable Deep Compressed Sensing. *arXiv* **2024**, arXiv:2308.13777.
37. Shafique, M.; Liu, S.; Schniter, P.; Ahmad, R. MRI recovery with self-calibrated denoisers without fully-sampled data. *Magn. Reson. Mater. Phys. Biol. Med.* **2024**, *38*, 53–66. [\[CrossRef\]](#)
38. Chen, Y.; Chen, X.; Jalali, S.; Maleki, A. Deep Memory Unrolled Networks for Solving Imaging Linear Inverse Problems. In Proceedings of the 15th International Conference on Sampling Theory and Applications, Vienna, Austria, 28 July–1 August 2025.
39. Kulkarni, K.; Lohit, S.; Turaga, P.; Kerviche, R.; Ashok, A. ReconNet: Non-Iterative Reconstruction of Images from Compressively Sensed Measurements. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 449–458. [\[CrossRef\]](#)
40. Sidky, E.Y.; Lorente, I.; Brankov, J.G.; Pan, X. Do CNNs Solve the CT Inverse Problem? *IEEE Trans. Biomed. Eng.* **2021**, *68*, 1799–1810. [\[CrossRef\]](#)
41. Gregor, K.; LeCun, Y. Learning fast approximations of sparse coding. In Proceedings of the 27th International Conference on Machine Learning, Madison, WI, USA, 24–26 November 2010; ICML'10, pp. 399–406.
42. Monga, V.; Li, Y.; Eldar, Y.C. Algorithm Unrolling: Interpretable, Efficient Deep Learning for Signal and Image Processing. *arXiv* **2020**, arXiv:1912.10557. [\[CrossRef\]](#)
43. Rezagah, F.E.; Jalali, S.; Erkip, E.; Poor, H.V. Compression-based compressed sensing. *IEEE Trans. Inf. Theory* **2017**, *63*, 6735–6752. [\[CrossRef\]](#)
44. Mardani, M.; Sun, Q.; Vasawanala, S.; Pappayan, V.; Monajemi, H.; Pauly, J.; Donoho, D. Neural Proximal Gradient Descent for Compressive Imaging. *arXiv* **2018**, arXiv:1806.03963. [\[CrossRef\]](#)
45. Li, Y.; Bar-Shira, O.; Monga, V.; Eldar, Y.C. Deep Algorithm Unrolling for Biomedical Imaging. *arXiv* **2021**, arXiv:2108.06637. [\[CrossRef\]](#)
46. Beck, A.; Teboulle, M. A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems. *SIAM J. Imaging Sci.* **2009**, *2*, 183–202. [\[CrossRef\]](#)
47. Zeng, C.; Yu, Y.; Wang, Z.; Xia, S.; Cui, H.; Wan, X. GSISTA-Net: Generalized structure ISTA networks for image compressed sensing based on optimized unrolling algorithm. *Multimed. Tools Appl.* **2024**, *83*, 80373–80387. [\[CrossRef\]](#)
48. Zhang, Z.; Liu, Y.; Liu, J.; Wen, F.; Zhu, C. AMP-Net: Denoising-Based Deep Unrolling for Compressive Image Sensing. *IEEE Trans. Image Process.* **2021**, *30*, 1487–1500. [\[CrossRef\]](#)
49. Georgescu, M.I.; Ionescu, R.T.; Verga, N. Convolutional Neural Networks With Intermediate Loss for 3D Super-Resolution of CT and MRI Scans. *IEEE Access* **2020**, *8*, 49112–49124. [\[CrossRef\]](#)

50. Mou, C.; Wang, Q.; Zhang, J. Deep Generalized Unfolding Networks for Image Restoration. In Proceedings of the 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 18–24 June 2022; pp. 17378–17389. [[CrossRef](#)]
51. Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; Wojna, Z. Rethinking the inception architecture for computer vision. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 2818–2826.
52. Zhang, K.; Zuo, W.; Chen, Y.; Meng, D.; Zhang, L. Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising. *IEEE Trans. Image Process.* **2017**, *26*, 3142–3155. [[CrossRef](#)] [[PubMed](#)]
53. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. *arXiv* **2015**, arXiv:1512.03385. [[CrossRef](#)]
54. He, K.; Zhang, X.; Ren, S.; Sun, J. Identity Mappings in Deep Residual Networks. *arXiv* **2016**, arXiv:1603.05027. [[CrossRef](#)]
55. Yamashita, R.; Nishio, M.; Do, R.K.G.; Togashi, K. Convolutional neural networks: An overview and application in radiology. *Insights Imaging* **2018**, *9*, 611–629. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.